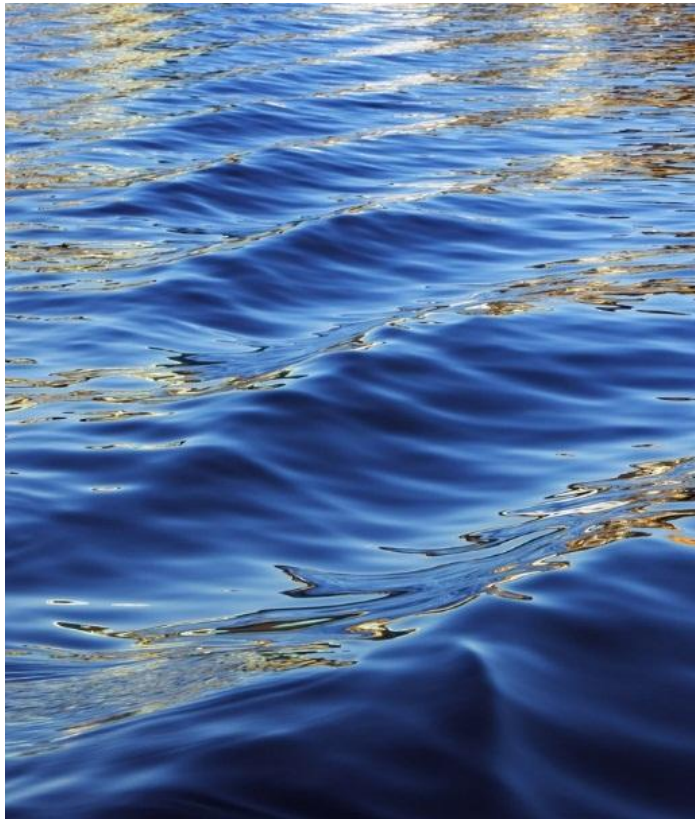
Python Language Basics

IoT 2

Champlain
COLLEGE SAINT-LAMBERT



The Python Interpreter

- Python is an interpreted language. The Python interpreter runs a program by executing one statement at a time. The standard interactive Python interpreter can be invoked on the command line with the python command:

```
$ python
Python 3.6.0 | packaged by conda-forge | (default, Jan 13 2017, 23:17:12)
[GCC 4.8.2 20140120 (Red Hat 4.8.2-15)] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> a = 5
>>> print(a)
5
```

- The >>> you see is the prompt where you'll type code expressions. To exit the Python interpreter and return to the command prompt, you can either type exit() or press Ctrl-D

Running a python program

- Running Python programs is as simple as calling python with a .py file as its first argument. Suppose we had created hello_world.py with these contents:

```
print('Hello world')
```

- You can run it by executing the following command (the hello_world.py file must be in your current working terminal directory):

```
$ python hello_world.py
Hello world
```

- While some Python programmers execute all of their Python code in this way, those doing data analysis or scientific computing make use of IPython, an enhanced Python interpreter, or Jupyter notebooks, web-based code notebooks originally created within the IPython project.

```
$ ipython
Python 3.6.0 | packaged by conda-forge | (default, Jan 13 2017, 23:17:12)
Type "copyright", "credits" or "license" for more information.

IPython 5.1.0 -- An enhanced Interactive Python.
?         -> Introduction and overview of IPython's features.
%quickref -> Quick reference.
help      -> Python's own help system.
object?   -> Details about 'object', use 'object??' for extra details.

In [1]: %run hello_world.py
Hello world

In [2]:
```

Language Semantics

- The Python language design is distinguished by its emphasis on readability, simplicity, and explicitness. Some people go so far as to liken it to "executable pseudocode."
- Indentation, not braces Python uses whitespace (tabs or spaces) to structure code instead of using braces as in many other languages like R, C++, Java, and Perl. Consider a for loop from a sorting algorithm:

```
for x in array:
    if x < pivot:
        less.append(x)
    else:
        greater.append(x)
```

- A colon denotes the start of an indented code block after which all of the code must be indented by the same amount until the end of the block.
- As you can see by now, Python statements also do not need to be terminated by semicolons. Semicolons can be used, however, to separate multiple statements on a single line:

```
a = 5; b = 6; c = 7
```

Everything is an object

- An important characteristic of the Python language is the consistency of its object model.
- Every number, string, data structure, function, class, module, and so on exists in the Python interpreter in its own "box," which is referred to as a Python object.
- Each object has an associated type (e.g., string or function) and internal data.
- In practice this makes the language very flexible, as even functions can be treated like any other object!

Comments

- Any text preceded by the hash mark (pound sign) # is ignored by the Python inter- preter. This is often used to add comments to code.

```
results = []
for line in file_handle:
    # keep the empty lines for now
    # if len(line) == 0:
    #   continue
    results.append(line.replace('foo', 'bar'))
```

- Comments can also occur after a line of executed code. While some programmers prefer comments to be placed in the line preceding a particular line of code, this can be useful at times

```
print("Reached this line")  # Simple status report
```

Function and object method calls

- You call functions using parentheses and passing zero or more arguments, optionally assigning the returned value to a variable:

```
result = f(x, y, z)
g()
```

- Almost every object in Python has attached functions, known as methods, that have access to the object's internal contents. You can call them using the following syntax:

```
obj.some_method(x, y, z)
```

- Functions can take both positional and keyword arguments:

```
result = f(a, b, c, d=5, e='foo')
```

More on this later

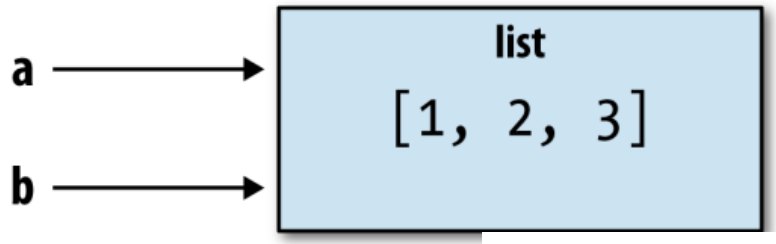
Variables and argument passing

- When assigning a variable (or name) in Python, you are creating a reference to the object on the righthand side of the equals sign. In practical terms, consider a list of integers:

```
In [8]: a = [1, 2, 3]
```

- Suppose we assign a to a new variable b. In some languages, this assignment would cause the data [1, 2, 3] to be copied. In Python, a and b actually now refer to the same object, the original list [1, 2, 3] (see Figure for a mockup).

```
In [9]: b = a
```



- You can prove this to yourself by appending an element to a and then examining b:

```
In [10]: a.append(4)

In [11]: b
Out[11]: [1, 2, 3, 4]
```

```
def append_element(some_list, element):
    some_list.append(element)
```

Then we have:

```
In [27]: data = [1, 2, 3]

In [28]: append_element(data, 4)

In [29]: data
Out[29]: [1, 2, 3, 4]
```

Dynamic references, strong types

- In contrast with many compiled languages, such as Java and C++, object references in Python have no type associated with them. There is no problem with the following:
- Variables are names for objects within a particular namespace; the type information is stored in the object itself. Some observers might hastily conclude that Python is not a "typed language." This is not true; consider this example:

```
In [12]: a = 5

In [13]: type(a)
Out[13]: int

In [14]: a = 'foo'

In [15]: type(a)
Out[15]: str
```

```
In [16]: '5' + 5
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-16-f9dbf5f0b234> in <module>()
----> 1 '5' + 5
TypeError: must be str, not int
```

Knowing the type of an object is important, and it's useful to be able to write functions that can handle many different kinds of input. You can check that an object is an instance of a particular type using the isinstance function:

```
In [21]: a = 5

In [22]: isinstance(a, int)
Out[22]: True
```

Attributes and methods

- Objects in Python typically have both attributes (other Python objects stored "inside" the object) and methods (functions associated with an object that can have access to the object's internal data). Both of them are accessed via the syntax obj.attribute_name

```
In [1]: a = 'foo'

In [2]: a.<Press Tab>
a.capitalize  a.format      a.isupper     a.rindex      a.strip
a.center      a.index       a.join        a.rjust       a.swapcase
a.count       a.isalnum     a.ljust       a.rpartition  a.title
a.decode      a.isalpha     a.lower       a.rsplit      a.translate
a.encode      a.isdigit     a.lstrip      a.rstrip      a.upper
a.endswith    a.islower     a.partition   a.split       a.zfill
a.expandtabs  a.isspace     a.replace     a.splitlines
a.find        a.istitle     a.rfind       a.startswith
```

Imports

- In Python a module is simply a file with the .py extension containing Python code. Suppose that we had the following module:

```
# some_module.py
PI = 3.14159

def f(x):
    return x + 2

def g(a, b):
    return a + b
```

- If we wanted to access the variables and functions defined in some_module.py, from another file in the same directory we could do:

```
import some_module
result = some_module.f(5)
pi = some_module.PI
```

Or equivalently:

```
from some_module import f, g, PI
result = g(5, PI)
```

- By using the as keyword you can give imports different variable names:

```
import some_module as sm
from some_module import PI as pi, g as gf

r1 = sm.f(pi)
r2 = gf(6, pi)
```

Binary operators and comparisons

- Most of the binary math operations and comparisons are as you might expect:

```
In [33]: 12 + 21.5
Out[33]: 33.5

In [34]: 5 <= 2
Out[34]: False
```

Operation	Description
`a + b`	Add a and b
`a - b`	Subtract b from a
`a * b`	Multiply a by b
`a / b`	Divide a by b
`a // b`	Floor-divide a by b, dropping any fractional remainder
`a ** b`	Raise a to the b power
`a & b`	`True` if both a and b are `True`; for integers, take the bitwise AND
`a | b`	`True` if either a or b is `True`; for integers, take the bitwise OR
`a ^ b`	For booleans, `True` if a or b is `True`, but not both; for integers, take the bitwise EXCLUSIVE-OR

Operation	Description
`a == b`	`True` if a equals b
`a != b`	`True` if a is not equal to b
`a <= b, a < b`	`True` if a is less than (less than or equal) to b
`a > b, a >= b`	`True` if a is greater than (greater than or equal) to b
`a is b`	`True` if a and b reference the same Python object
`a is not b`	`True` if a and b reference different Python objects

Comparing references

- To check if two references refer to the same object, use the **is** keyword. **is not** is also perfectly valid if you want to check that two objects are not the same:

```
In [35]: a = [1, 2, 3]

In [36]: b = a

In [37]: c = list(a)

In [38]: a is b
Out[38]: True

In [39]: a is not c
Out[39]: True
```

- Since list always creates a new Python list (i.e., a copy), we can be sure that c is distinct from a. Comparing with **is** is not the same as the == operator, because in this case we have:

```
In [40]: a == c
Out[40]: True
```

- A very common use of **is** and **is not** is to check if a variable is None, since there is only one instance of None:

```
In [41]: a = None

In [42]: a is None
Out[42]: True
```

Scalar Types

- python along with its standard library has a small set of built-in types for handling numerical data, strings, boolean (True or False) values, and dates and time. These "single value" types are sometimes called scalar types and we refer to them in this book as scalars.

Type	Description
`None`	The Python "null" value (only one instance of the `None` object exists)
`str`	String type; holds Unicode (UTF-8 encoded) strings
`bytes`	Raw ASCII bytes (or Unicode encoded as bytes)
`float`	Double-precision (64-bit) floating-point number (note there is no separate `double` type)
`bool`	A `True` or `False` value
`int`	Arbitrary precision signed integer

Numeric types

- The primary Python types for numbers are int and float. An int can store arbitrarily large numbers:

```
In [48]: ival = 17239871

In [49]: ival ** 6
Out[49]: 26254519291092456596965462913230729701102721
```

- Floating-point numbers are represented with the Python float type. Under the hood each one is a double-precision (64-bit) value. They can also be expressed with scientific notation:

```
In [50]: fval = 7.243

In [51]: fval2 = 6.78e-5
```

- Integer division not resulting in a whole number will always yield a floating-point number:

```
In [52]: 3 / 2
Out[52]: 1.5
```

- To get C-style integer division (which drops the fractional part if the result is not a whole number), use the floor division operator //:

```
In [53]: 3 // 2
Out[53]: 1
```

Strings

- Many people use Python for its powerful and flexible built-in string processing capabilities. You can write string literals using either single quotes ' or double quotes ":

```
a = 'one way of writing a string'
b = "another way"
```

- For multiline strings with line breaks, you can use triple quotes, either ''' or """

```
c = """
This is a longer string that
spans multiple lines
"""
```

- Many Python objects can be converted to a string using the str function:

```
In [61]: a = 5.6

In [62]: s = str(a)

In [63]: print(s)
5.6
```

Strings are a sequence

- Strings are a sequence of Unicode characters and therefore can be treated like other sequences, such as lists and tuples (which we will explore in more detail in the next chapter):

```
In [64]: s = 'python'

In [65]: list(s)
Out[65]: ['p', 'y', 't', 'h', 'o', 'n']

In [66]: s[:3]
Out[66]: 'pyt'
```

- The backslash character \ is an escape character, meaning that it is used to specify special characters like newline \n or Unicode characters. To write a string literal with backslashes, you need to escape them:

```
In [67]: s = '12\\34'

In [68]: print(s)
12\34
```

- Adding two strings together concatenates them

and produces a new string:

```
In [69]: s = r'this\has\no\special\characters'

In [70]: s
Out[70]: 'this\\has\\no\\special\\characters'
```

The r stands for *raw*.

strings are immutable

- Python strings are immutable; you cannot modify a string:

```
In [56]: a = 'this is a string'

In [57]: a[10] = 'f'
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-57-5ca625d1e504> in <module>()
----> 1 a[10] = 'f'
TypeError: 'str' object does not support item assignment

In [58]: b = a.replace('string', 'longer string')

In [59]: b
Out[59]: 'this is a longer string'
```

After this operation, the variable a is unmodified:

```
In [60]: a
Out[60]: 'this is a string'
```

Type casting

- The str, bool, int, and float types are also functions that can be used to cast values to those types:

```
In [91]: s = '3.14159'

In [92]: fval = float(s)

In [93]: type(fval)
Out[93]: float

In [94]: int(fval)
Out[94]: 3

In [95]: bool(fval)
Out[95]: True

In [96]: bool(0)
Out[96]: False
```

None

- None is the Python null value type.
- If a function does not explicitly return a value, it implicitly returns None.

- None is also a common default value for function arguments:

```
In [97]: a = None

In [98]: a is None
Out[98]: True

In [99]: b = 5

In [100]: b is not None
Out[100]: True
```

Dates and times

- The built-in Python datetime module provides datetime, date, and time types. The datetime type, as you may imagine, combines the information stored in date and time and is the most commonly used:

```
In [102]: from datetime import datetime, date, time

In [103]: dt = datetime(2011, 10, 29, 20, 30, 21)

In [104]: dt.day
Out[104]: 29

In [105]: dt.minute
Out[105]: 30
```

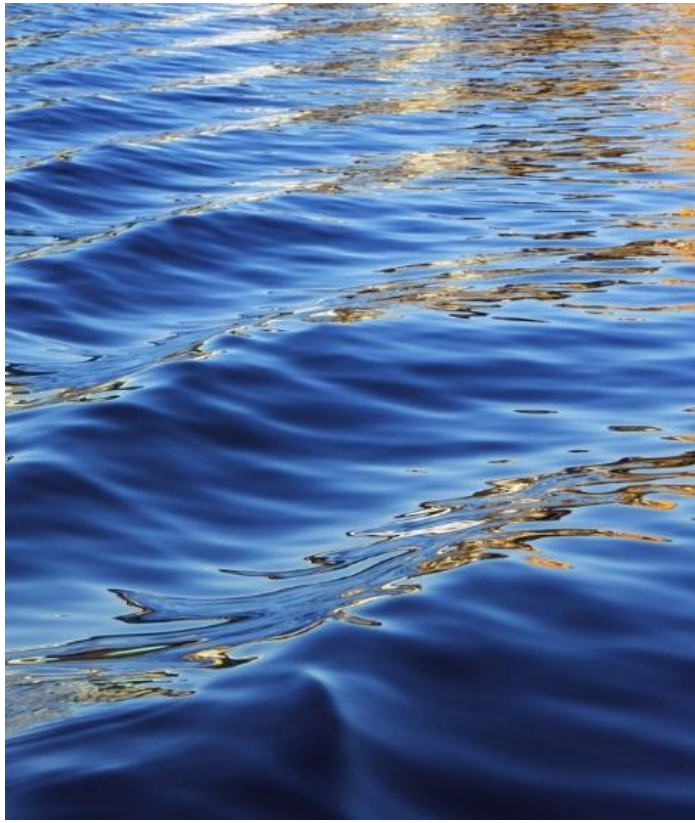
- Given a datetime instance, you can extract the equivalent date and time objects by calling methods on the datetime of the same name:

```
In [106]: dt.date()
Out[106]: datetime.date(2011, 10, 29)

In [107]: dt.time()
Out[107]: datetime.time(20, 30, 21)
```

Champlain
COLLEGE SAINT-LAMBERT

Control flow



if, elif, and else

- The if statement is one of the most well-known control flow statement types. It checks a condition that, if True, evaluates the code in the block that follows:

```
if x < 0:
    print('It's negative')
```

- An if statement can be optionally followed by one or more elif blocks and a catch all else block if all of the conditions are False. If any of the conditions is True, no further elif or else blocks will be reached. With a compound condition using and or or, conditions are evaluated left to right and will short-circuit:

```
if x < 0:
    print('It's negative')
elif x == 0:
    print('Equal to zero')
elif 0 < x < 5:
    print('Positive but smaller than 5')
else:
    print('Positive and larger than or equal to 5')
```

for loops

- for loops are for iterating over a collection (like a list or tuple) or an iterater. The standard syntax for a for loop is

```
for value in collection:
    # do something with value
```

- You can advance a for loop to the next iteration, skipping the remainder of the block, using the continue keyword. Consider this code, which sums up integers in a list and skips None values:

```
sequence = [1, 2, None, 4, None, 5]
total = 0
for value in sequence:
    if value is None:
        continue
    total += value
```

- A for loop can be exited altogether with the break keyword. This code sums elements of the list until a 5 is reached:

```
sequence = [1, 2, 0, 4, 6, 5, 2, 1]
total_until_5 = 0
for value in sequence:
    if value == 5:
        break
    total_until_5 += value
```

while loops

- A while loop specifies a condition and a block of code that is to be executed until the condition evaluates to False or the loop is explicitly ended with break:

```
x = 256
total = 0
while x > 0:
    if total > 500:
        break
    total += x
    x = x // 2
```

- pass is the "no-op" statement in Python. It can be used in blocks where no action is to be taken (or as a placeholder for code not yet implemented); it is only required because Python uses whitespace to delimit blocks:

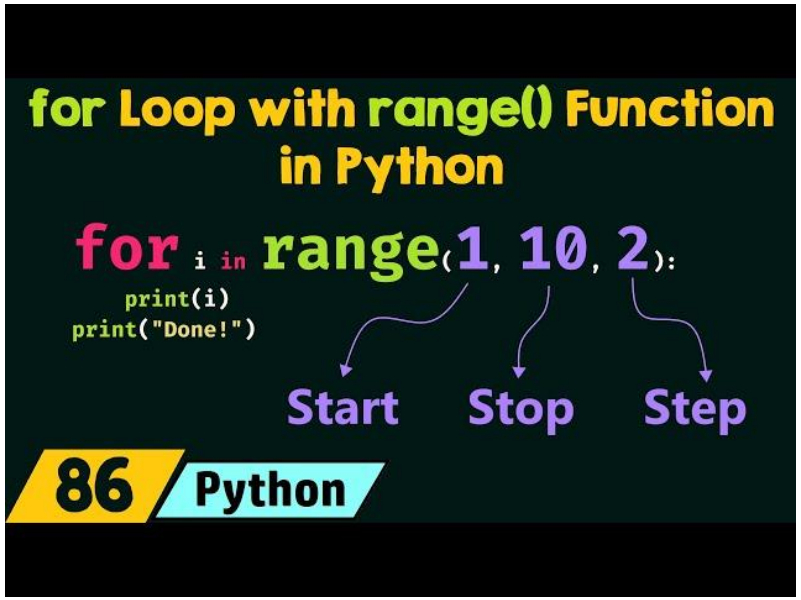
```
if x < 0:
    print('negative!')
elif x == 0:
    # TODO: put something smart here
    pass
else:
    print('positive!')
```

range

- The range function returns an iterator that yields a sequence of evenly spaced integers. Both a start, end, and step (which may be negative) can be given:

```
In [124]: list(range(0, 20, 2))
Out[124]: [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

In [125]: list(range(5, 0, -1))
Out[125]: [5, 4, 3, 2, 1]
```



- As you can see, range produces integers up to but not including the endpoint. A common use of range is for iterating through sequences by index:

```
seq = [1, 2, 3, 4]
for i in range(len(seq)):
    val = seq[i]
```

for loops

- The break keyword only terminates the innermost for loop; any outer for loops will continue to run:

```
In [121]: for i in range(4):
   .....:     for j in range(4):
   .....:         if j > i:
   .....:             break
   .....:         print((i, j))
   .....:
(0, 0)
(1, 0)

(1, 1)
(2, 0)
(2, 1)
(2, 2)
(3, 0)
(3, 1)
(3, 2)
(3, 3)
```

- As we will see in more detail, if the elements in the collection or iterator are sequences (tuples or lists, say), they can be conveniently unpacked into variables in the for loop statement:

```
for a, b, c in iterator:
    # do something
```

range

- While you can use functions like list to store all the integers generated by range in some other data structure, often the default iterator form will be what you want. This snippet sums all numbers from 0 to 99,999 that are multiples of 3 or 5:

```
sum = 0
for i in range(100000):
    # % is the modulo operator
    if i % 3 == 0 or i % 5 == 0:
        sum += i
```

- While the range generated can be arbitrarily large, the memory use at any given time may be very small.

Ternary expressions

- A ternary expression in Python allows you to combine an if-else block that produces a value into a single line or expression. The syntax for this in Python is:

```
value = true-expr if condition else false-expr
```

- Here, true-expr and false-expr can be any Python expressions. It has the identical effect as the more verbose:

```
if condition:
    value = true-expr
else:
    value = false-expr
```

- This is a more concrete example:

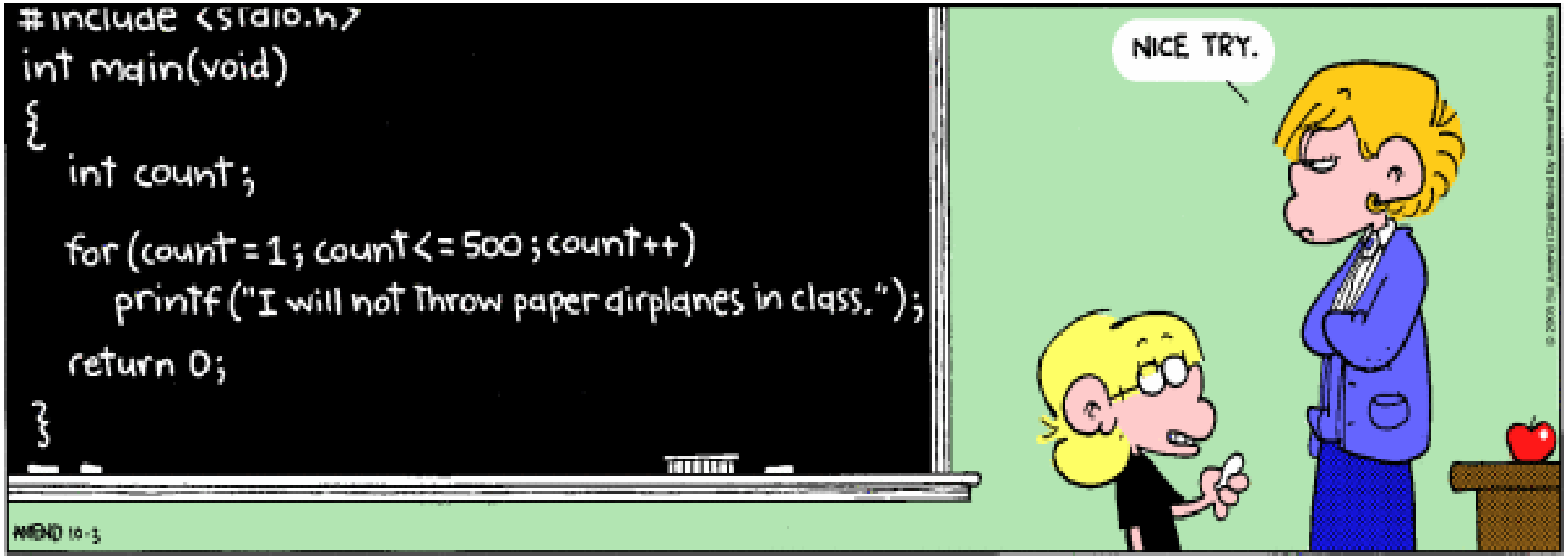
```
In [126]: x = 5

In [127]: 'Non-negative' if x >= 0 else 'Negative'
Out[127]: 'Non-negative'
```

The benefits of loops

- Sometime ,you need to write something many times for some useful reason.

So we might as well use cleverness to do it. Th*at's what loops are for.*



It doesn't have to be the exact same thing over and over.

And this is how we really harness the power of a computer that can perform tens of billions (or more) computations per second!

Complete lab-01